Reporte y Explicación Práctica 15

En este programa desarrollamos una calculadora de fractales utilizando el paradigma de Programación Orientada a Objetos (POO) y la técnica de recursividad. Creamos una clase llamada Fractales, en la cual agrupamos los métodos necesarios para calcular el Triángulo de Sierpinski y el Polvo de Cantor. El programa permite al usuario seleccionar el fractal que desea analizar, ingresar el número de iteraciones y establecer una longitud inicial para obtener la cantidad de triángulos o segmentos generados, así como sus respectivas longitudes. Para realizar estos cálculos utilizamos funciones recursivas, que permiten repetir operaciones matemáticas mediante llamadas a sí mismas hasta alcanzar una condición de finalización. De esta manera, aplicamos los conceptos de clases, objetos, encapsulamiento y recursividad para organizar y resolver los cálculos de ambos fractales.

Código Completo:

```
#include <iostream>
#include <cmath>
using namespace std;

double potencia(double base, int exponente) {
    if (exponente == 0) {
        return 1;
    }

    return base * potencia(base, exponente - 1);
}

unsigned long long triangulosSierpinski(int n) {
    if (n == 0) {
        return 1;
    }

    return 3 * triangulosSierpinski(n - 1);
}

unsigned long long segmentosCantor(int n) {
    if (n == 0) {
        return 1;
    }

    return 2 * segmentosCantor(n - 1);
}

int main() {
    int opcion, n;
    double L;

    do {
        cout << "\n===== MENU DE FRACTALES =====" << endl;
        cout << "1. Triangulo de Sierpinski" << endl;
        cout << "2. Polvo de Cantor" << endl;
        cout << "3. Salir" << endl;
        cout << "Elige una opcion: ";
        cin >> opcion;

        if (opcion == 1) {
            cout << "\nTRIANGULO DE SIERPINSKI" << endl;
```

```
            cout << "Numero de iteraciones (0 a 39): ";
            cin >> n;

            if (n < 0 || n > 39) {
                cout << "Iteraciones no validas." << endl;
                continue;
            }



            unsigned long long cantidad =
                triangulosSierpinski(n);

            double lado = L / potencia(2, n);


            cout << "\n--- RESULTADOS ---" << endl;
            cout << "Iteraciones: " << n << endl;
            cout << "Triangulos creados: " << cantidad << endl;

        }
        else if (opcion == 2) {
            cout << "\nPOLVO DE CANTOR" << endl;
            cout << "Numero de iteraciones (0 a 60): ";
            cin >> n;

            if (n < 0 || n > 60) {
                cout << "Iteraciones no validas." << endl;
                continue;
            }



            unsigned long long cantidad =
                segmentosCantor(n);

            double longitud = L / potencia(3, n);
            double total = L * potencia(2.0 / 3.0, n);

            cout << "\n--- RESULTADOS ---" << endl;
            cout << "Iteraciones: " << n << endl;
            cout << "Segmentos creados: " << cantidad << endl;

        }
        else if (opcion == 3) {
            cout << "Programa terminado." << endl;
        }
        else {
            cout << "Opcion no valida." << endl;
        }

    } while (opcion != 3);

    return 0;
}
```

Explicación del Código:

1. Definición de la Clase (class Fractales)

Esta clase reúne las operaciones matemáticas y el menú principal del programa.

2. Declaración del Atributo (private: double L)

Dentro de esta sección declaramos la variable L, que almacena la longitud inicial del fractal ingresada por el usuario. Utilizamos private: para proteger este atributo y evitar que pueda modificarse directamente desde fuera de la clase. Su valor se utiliza posteriormente para calcular las longitudes de los lados y segmentos después de cada iteración.

3. Función Recursiva para Calcular Potencias (double potencia(double base, int exponente))

Es un método recursivo encargado de calcular una potencia sin utilizar directamente la función pow(). Recibe dos parámetros: base, que representa el número que se elevará, y exponente, que indica cuántas veces se multiplicará por sí mismo.

Primero utiliza una condición if para establecer el caso base: cuando el exponente es igual a cero, devuelve 1. Si el exponente es mayor que cero, multiplica la base por el resultado de volver a llamar a la misma función con el exponente disminuido en uno. De esta manera, la función se repite hasta alcanzar el caso base.

4. Función Recursiva para Contar Triángulos (unsigned long long triangulosSierpinski(int n))

Se encarga de calcular la cantidad de triángulos que se generan en el Triángulo de Sierpinski dependiendo del número de iteraciones ingresado. Utiliza el parámetro n para representar las iteraciones.

Primero verifica si n es igual a cero; si es así, devuelve 1, porque inicialmente existe un solo triángulo. Si el número de iteraciones es mayor que cero, multiplica por 3 el resultado de volver a llamar a la función con n - 1. De esta manera, en cada iteración se triplica la cantidad de triángulos.

Utilizamos unsigned long long porque permite almacenar números enteros grandes sin utilizar valores negativos.

5. Función Recursiva para Contar Segmentos (unsigned long long segmentosCantor(int n))

Es el método encargado de calcular la cantidad de segmentos que se generan en el Polvo de Cantor. Funciona de manera similar a la función anterior, pero en este caso la cantidad de segmentos se duplica en cada iteración.

Primero establece que, cuando n es igual a cero, existe un solo segmento. Después, si todavía quedan iteraciones, multiplica por 2 el resultado de llamar nuevamente a la función con n - 1. Así obtenemos la cantidad total de segmentos generados.

6. Método para Calcular el Triángulo de Sierpinski (void calcularSierpinski())

Es el método encargado de realizar los cálculos correspondientes al Triángulo de Sierpinski. Primero solicita al usuario el número de iteraciones y verifica que se encuentre entre 0 y 39. Si el valor no está dentro del rango permitido, muestra un mensaje de error y termina el método mediante return.

Después solicita la longitud inicial y utiliza la función recursiva triangulosSierpinski(n) para obtener la cantidad de triángulos generados. También utiliza la función potencia(2, n) para calcular la longitud de cada lado después de las iteraciones, dividiendo la longitud inicial entre el resultado obtenido.

Finalmente, muestra el número de iteraciones, la cantidad de triángulos creados y la longitud de cada lado.

7. Método para Calcular el Polvo de Cantor (void calcularCantor())

Se encarga de realizar los cálculos correspondientes al Polvo de Cantor. Primero solicita el número de iteraciones y verifica que se encuentre entre 0 y 60. Si el valor no es válido, muestra un mensaje de error y termina el método.

Posteriormente solicita la longitud inicial y utiliza la función recursiva segmentosCantor(n) para determinar cuántos segmentos existen después de las iteraciones.

También realiza dos cálculos adicionales:

- longitud = L / potencia(3, n): calcula la longitud de cada segmento después de las iteraciones.
- total = L * potencia(2.0 / 3.0, n): calcula la longitud total que se conserva después de eliminar el tercio central en cada iteración.

Finalmente, muestra el número de iteraciones, la cantidad de segmentos creados, la longitud de cada segmento y la longitud total conservada.

8. Método para Mostrar el Menú (void menu())

Es el método encargado de presentar las opciones disponibles del programa. Utiliza un ciclo do-while para mostrar el menú repetidamente hasta que el usuario seleccione la opción 3 para salir.

Dentro del menú utilizamos una estructura switch(opcion) para identificar la selección del usuario. Dependiendo de la opción elegida, se llama al método calcularSierpinski() para calcular el primer fractal, calcularCantor() para calcular el segundo o se termina el programa. También incluye una opción default para indicar cuando se selecciona una opción no válida.

9. Instanciación del Objeto (Fractales calculadora;)

Creamos un objeto llamado calculadora basado en nuestra clase Fractales. Este objeto nos permite acceder a los métodos públicos de la clase y ejecutar las diferentes operaciones del programa sin necesidad de acceder directamente a sus atributos privados.

10. Llamada al Método Principal (calculadora.menu())

Utilizamos el operador punto (.) para acceder al método menu() del objeto calculadora. Al ejecutarlo, se muestra el menú principal y se permite al usuario seleccionar el fractal que desea calcular. A partir de esta selección se ejecutan los métodos correspondientes.

11. Función Principal (int main())

Es el punto de entrada del programa. En este caso, su función es crear el objeto calculadora y llamar a su método menu(), que controla el funcionamiento de todo el sistema. De esta manera, evitamos colocar todas las instrucciones directamente dentro de main() y permitimos que la clase Fractales se encargue de realizar las operaciones correspondientes.